Assignment – 6:

1. Create a React component called ProductList that renders a list of 1000 product names and prices, and measure how long it takes to render the list in your browser.

2. Update your ProductList to include a filter input that lets users type a search term to filter products by name, and use the useMemo() hook to memoize the filtered list so it only recalculates when the search term or product list changes.

Hint: Place the filtering logic inside useMemo and pass [products, searchTerm] as dependencies.

3. Build a React component called PlaylistManager that displays a list of songs and lets users mark songs as favorite. Use the useCallback() hook to memoize the function that toggles a song's favorite status, and pass it down to each SongItem component.

4. Given a React component that rerenders a large list every time a button is clicked, refactor the code to use useMemo() and useCallback() to minimize unnecessary rerenders and improve performance.

Hint: Profile before and after using React DevTools and note the difference.

Answer : - 1. ProductList - Render 1000 Products and Measure Render Time

```
import React, { useEffect } from "react";

const ProductList = () => {
    const products = Array.from({ length: 1000 }, (_, index) => ({
        id: index + 1,
        name: `Product ${index + 1}`,
        price: Math.floor(Math.random() * 1000),
    }));

    useEffect(() => {
        console.time("Render Time");

        return () => {
            console.timeEnd("Render Time");
        };
    }, []);

    return (
        <div>
            <h2>Product List</h2>

            {products.map((product) => (
                <div key={product.id}>
```

```
                    <p>
                        {product.name} - ₹{product.price}
                    </p>
                </div>
            ))}
        </div>
    );
};

export default ProductList;
```

2. ProductList with Search + useMemo

```
import React, { useMemo, useState } from "react";

const ProductList = () => {
    const [searchTerm, setSearchTerm] = useState("");

    const products = Array.from({ length: 1000 }, (_, index) => ({
        id: index + 1,
        name: `Product ${index + 1}`,
        price: Math.floor(Math.random() * 1000),
    }));

    const filteredProducts = useMemo(() => {
        console.log("Filtering Products...");

        return products.filter((product) =>
            product.name.toLowerCase().includes(searchTerm.toLowerCase())
        );
    }, [products, searchTerm]);

    return (
        <div>
            <h2>Search Product</h2>

            <input
                type="text"
                placeholder="Search Product"
                value={searchTerm}
                onChange={(e) => setSearchTerm(e.target.value)}
            />

            {filteredProducts.map((product) => (
                <div key={product.id}>
```

```
                    <p>
                        {product.name} - ₹{product.price}
                    </p>
                </div>
            ))}
        </div>
    );
};

export default ProductList;
```

3. PlaylistManager using useCallback

```
import React from "react";

const SongItem = ({ song, toggleFavorite }) => {
    console.log("Rendering:", song.name);

    return (
        <div>
            <span>
                {song.name} {song.favorite ? "🩶" : "♡"}
            </span>

            <button onClick={() => toggleFavorite(song.id)}>
                Toggle Favorite
            </button>
        </div>
    );
};

export default React.memo(SongItem);
```

PlaylistManager.jsx

```
import React, { useCallback, useState } from "react";
import SongItem from "./SongItem";

const PlaylistManager = () => {
    const [songs, setSongs] = useState([
        { id: 1, name: "Shape of You", favorite: false },
        { id: 2, name: "Believer", favorite: false },
        { id: 3, name: "Perfect", favorite: false },
    ]);

    const toggleFavorite = useCallback((id) => {
        setSongs((prevSongs) =>
```

```
            prevSongs.map((song) =>
                song.id === id
                    ? { ...song, favorite: !song.favorite }
                    : song
            )
        );
    }, []);

    return (
        <div>
            <h2>Playlist Manager</h2>

            {songs.map((song) => (
                <SongItem
                    key={song.id}
                    song={song}
                    toggleFavorite={toggleFavorite}
                />
            ))}
        </div>
    );
};

export default PlaylistManager;
```

4. Optimizing Large List using useMemo + useCallback

Before Optimization

```
import React, { useState } from "react";

const App = () => {
    const [count, setCount] = useState(0);

    const products = Array.from({ length: 1000 }, (_, index) => ({
        id: index,
        name: `Product ${index}`,
    }));

    return (
        <div>
            <button onClick={() => setCount(count + 1)}>
                Count: {count}
            </button>

            {products.map((item) => (
                <p key={item.id}>{item.name}</p>
```

```
            ))}
        </div>
    );
};

export default App;
```

After Optimization

```
import React from "react";

const ProductItem = ({ product, handleClick }) => {
    console.log("Rendering:", product.name);

    return (
        <div>
            <p>{product.name}</p>

            <button onClick={() => handleClick(product.id)}>
                Select
            </button>
        </div>
    );
};

export default React.memo(ProductItem);
```

App.jsx

```
import React, {
    useCallback,
    useMemo,
    useState,
} from "react";

import ProductItem from "./ProductItem";

const App = () => {
    const [count, setCount] = useState(0);

    const products = useMemo(() => {
        return Array.from({ length: 1000 }, (_, index) => ({
            id: index,
            name: `Product ${index}`,
        }));
```

```
    }, []);

    const handleClick = useCallback((id) => {
        console.log("Clicked:", id);
    }, []);

    return (
        <div>
            <button onClick={() => setCount(count + 1)}>
                Count: {count}
            </button>

            {products.map((product) => (
                <ProductItem
                    key={product.id}
                    product={product}
                    handleClick={handleClick}
                />
            ))}
        </div>
    );
};

export default App;
```